Week 1 - Domain 1: SDLC Automation - AWS CodePipeline

Study week: Oct 19 - Oct 25, 2026 **Exam:** AWS DOP-C02 (Domain 1 = 22% of the exam)

Section	What it covers
5.1 - 5.3	Pipeline structure, action types and providers, change detection
5.4 - 5.5	Artifacts in S3 + KMS, manual approvals
5.6 - 5.7	Cross-Region and cross-account pipelines
5.8 - 5.9	Variables, pipeline type V2 (triggers, execution modes, conditions, rollback)
5.10 - 5.11	Deploy actions in detail, monitoring and control
5.12 - 5.14	Exam traps, hands-on lab, check yourself

Part 5: AWS CodePipeline

5.1 What CodePipeline is

- **A managed continuous-delivery service.** It **coordinates** the steps from source to build, test, approval and deployment. It doesn't build or deploy anything itself; it **calls other services** (CodeBuild, CodeDeploy, CloudFormation, ECS, Lambda) or third-party tools (Jenkins, GitHub).

Part	What it is
Pipeline	The whole workflow, defined as JSON / YAML (or CloudFormation, CDK)
Stage	A group of actions, e.g. Source > Build > Test > Approve > Deploy-Prod. Each stage works on one execution at a time.
Action	One task inside a stage, e.g. "CodeBuild project X" or "Deploy stack Y"
Transition	The link between two stages. Can be disabled to stop changes moving forward.
Execution	One run of the pipeline for one set of source changes, with its own execution ID
Artifact	Files passed between actions, stored zipped in S3

- **runOrder:** actions with the **same runOrder run in parallel**; a higher runOrder runs **after** the lower ones finish.
- **A stage fails if any action in it fails.** You can **retry the failed actions** or **retry the whole stage** without re-running earlier stages.

5.2 Action types and providers

Category	Providers to know
Source	CodeCommit, S3 (bucket **versioning** required), ECR (image push), CodeConnections (formerly CodeStar Connections) for GitHub, GitHub Enterprise, Bitbucket, GitLab
Build	CodeBuild, Jenkins, Commands action (V2), ECRBuildAndPublish (newer)
Test	CodeBuild, Device Farm, third-party (BlazeMeter, Ghost Inspector), InspectorScan (newer)
Deploy	CodeDeploy (EC2 / on-premises, CodeDeployToECS blue/green), ECS (rolling), Elastic Beanstalk, CloudFormation, CloudFormation StackSets, S3 (can extract), AppConfig, Service Catalog, newer EKS and Lambda deploy actions
Approval	Manual approval
Invoke	Lambda, Step Functions, Snyk

- **GitHub through CodeConnections** is the current method. The old "GitHub version 1" (OAuth token) action is **deprecated**. The pipeline role needs `codestar-connections:UseConnection` (or `codeconnections:UseConnection`).
- **Output artifact format** (CodeCommit / CodeConnections):

  - `CODE_ZIP` (default) - a ZIP of the files, **without** Git history.
  - `CODEBUILD_CLONE_REF` - CodeBuild does a **full git clone** (history, tags, submodules). The **CodeBuild role** then needs `codecommit:GitPull` or `codestar-connections:UseConnection`.

5.3 How the pipeline notices a change

Source	Recommended way
CodeCommit	EventBridge rule on referenceUpdated (PollForSourceChanges = false)
S3	EventBridge (S3 event notifications to EventBridge, or a CloudTrail trail logging PutObject data events)
ECR	EventBridge "ECR Image Action" push event
GitHub / GitLab / Bitbucket	The connection's webhook, configured with V2 triggers (5.9)

- **Polling** (`PollForSourceChanges: true`) is **legacy** - slow and many API calls. "Start within seconds" = EventBridge or webhook.
- **S3 source:** the bucket needs **versioning**; the pipeline tracks the object version.

5.4 Artifacts in S3 and KMS

- Each pipeline has an **artifact store**: an **S3 bucket in the pipeline's Region**. Actions read **input artifacts** and write **output artifacts**, referenced by name (`SourceOutput` > `BuildOutput`). Artifacts are zipped.
- **CodePipeline never deletes old artifacts** - add an **S3 lifecycle rule** to control cost.
- **Encryption:**
  - Default: the **AWS-managed** `aws/s3` **key** - works **within one account**.
  - **Cross-account pipelines need a customer-managed KMS key**, because only its key policy can allow other accounts.
  - Set in `artifactStore.encryptionKey` (type KMS, key ARN / ID).
- **Securing the bucket:** block public access; bucket policy that **denies unencrypted uploads** and **denies non-HTTPS** requests (`aws:SecureTransport = false`); for cross-account, grant the other account's role `s3:GetObject*`, `s3:PutObject`, `s3:ListBucket`.
- **Service role** needs bucket access, `kms:Encrypt / Decrypt / GenerateDataKey* / ReEncrypt* / DescribeKey`, and permission to call each action's service.

5.5 Manual approvals

- An **Approval** action **pauses** the pipeline until someone approves or rejects.
- Settings: **SNS topic** (optional, to email / Slack / Lambda), **external review URL** (optional), **comments**.
- **Timeout: 7 days.** No response = the action **fails**.
- Reject = action fails. Approve = pipeline continues. Who approved and comments are recorded (also in CloudTrail).
- **Restrict approvers** to one specific action with IAM:

```
{
  "Effect": "Allow",
  "Action": "codepipeline:PutApprovalResult",
  "Resource": "arn:aws:codepipeline:us-east-1:111122223333:MyPipeline/Prod-Approval/ApproveRelease"
}
```

  - Also give `codepipeline:GetPipeline*` and `ListPipelines` so approvers can see the pipeline.
- **Automating approvals:** a Lambda function reads the approval **token** with `GetPipelineState`, then calls `PutApprovalResult` (e.g. from a Slack button or a ticket check).
- **Exam patterns:**
  - "Human must approve before production" = **manual approval action** before the prod deploy stage, with SNS.

  - "Review infrastructure change before applying" = **CHANGE_SET_REPLACE** > manual approval > **CHANGE_SET_EXECUTE**.
  - "Pause all deployments during a freeze" = **disable the inbound transition** to prod (V2: **DeploymentWindow** condition).

5.6 Cross-Region pipelines

- One pipeline can run actions in **other Regions** (each action has a `region` setting).
- **Source actions must be in the pipeline's own Region.**
- **Every Region used needs its own artifact bucket** - `artifactStores` (a map) instead of `artifactStore`.
- CodePipeline **copies artifacts automatically** to the other Regions' buckets.
- **KMS:** each Region's bucket needs a **key in that Region** (or multi-Region keys).

```
"artifactStores": {
  "us-east-1": { "type": "S3", "location": "pipeline-artifacts-use1",
                 "encryptionKey": { "id": "arn:aws:kms:us-east-1:111122223333:key/aaa", "type": "KMS" } },
  "eu-west-1": { "type": "S3", "location": "pipeline-artifacts-euw1",
                 "encryptionKey": { "id": "arn:aws:kms:eu-west-1:111122223333:key/bbb", "type": "KMS" } }
}
```

- The **console** creates the Region buckets for you; with **CloudFormation / CLI** you create them yourself.

5.7 Cross-account pipelines

Typical setup: pipeline in a **Tools / DevOps account**, deploying to **Dev, Test, Prod accounts**.

#	Where	What
1	Tools	Customer-managed KMS key for the artifact bucket; key policy allows the target accounts' roles
2	Tools	Artifact bucket policy allowing the target accounts' roles to get / put objects
3	Target (e.g. Prod)	Cross-account role that trusts the Tools account, with permissions for the action (CloudFormation, CodeDeploy, ECS...) plus S3 and KMS access to the Tools bucket / key
4	Tools	Pipeline service role gets sts:AssumeRole on that target role
5	Pipeline definition	The action's roleArn = the target account's role
6	Target (CloudFormation)	A CloudFormation deployment role (RoleArn in the action configuration) that creates the resources; the cross-account role needs iam:PassRole on it

- **Source in another account** (CodeCommit in Dev, pipeline in Tools): a role in Dev the pipeline assumes for the source action, plus an **EventBridge rule in Dev sending CodeCommit events to the Tools event bus**, and a rule in Tools that starts the pipeline.
- Cross-account actions **can't be fully set up in the console** - use CLI, CloudFormation or CDK. **CDK Pipelines** does it for you (`crossAccountKeys: true`).
- **Exam traps:**
  - Cross-account deploy fails with **KMS AccessDenied / can't decrypt artifact** = default `aws/s3` key. Use a **customer-managed key** and allow the target account.
  - Target role can't read artifacts = fix the **bucket policy and key policy** in Tools.

5.8 Variables

Syntax: `#{namespace.variable}`. Values are resolved when each action runs and are **specific to each execution** (safer than writing them to Parameter Store).

Variable type	Example	Notes
Action output variables	#{SourceVariables.CommitId}, #{SourceVariables.BranchName}, #{SourceVariables.CommitMessage}	Give the action a namespace; each provider outputs its own set

Variable type	Example	Notes
CodeBuild exported variables	#{BuildVariables.IMAGE_TAG}	exported-variables in the buildspec
Lambda output variables	#{LambdaVars.ticket}	outputVariables in PutJobSuccessResult
CloudFormation stack outputs	#{DeployVars.ApiUrl}	Stack outputs become variables
Pipeline-level variables (V2)	#{variables.TargetEnv}	Defaults on the pipeline, can be overridden when the pipeline is started
Built-in	#{codepipeline.PipelineExecutionId}	Always available

- Uses: pass commit ID / image tag from build to deploy; put the commit message in the approval comment; choose the environment when starting manually.

5.9 Pipeline type V2

	V1	V2
Pricing	About $1 per active pipeline per month	Per action execution minute (with free monthly minutes)
Triggers with tag / branch / file path / pull request filters	No	Yes
Pipeline-level variables	No	Yes
Execution modes	SUPERSEDED only	SUPERSEDED, QUEUED, PARALLEL
Stage conditions, automatic rollback and retry	No	Yes
Commands action	No	Yes

- A V1 pipeline **can be upgraded** to V2 in place.

Triggers (CodeConnections sources: GitHub, GitLab, Bitbucket):

Trigger type	Filters
Push	Branches (include / exclude, wildcards like release*), file paths (include / exclude, e.g. services/payments/**), Git tags (e.g. v*)
Pull request	Events opened / updated / closed; filters on target branches and file paths

```
"triggers": [{
  "providerType": "CodeStarSourceConnection",
  "gitConfiguration": {
    "sourceActionName": "Source",
    "push": [{
      "branches":  { "includes": ["main", "release/*"] },
      "filePaths": { "includes": ["services/payments/**"], "excludes": ["**/*.md"] }
    }],
    "pullRequest": [{
      "events": ["OPEN", "UPDATED"],
      "branches": { "includes": ["main"] }
    }]
  }
}]
```

- Monorepo, run only when one folder changes = **push trigger with a file path filter**.
- Release only on a v* tag = **Git tag trigger**.
- Validate pull requests into main = **pull request trigger**.
- V1 has none of these filters (custom EventBridge + Lambda needed).

Execution modes:

Mode	Behaviour	Use it when
SUPERSEDED (default, V1 + V2)	A newer execution replaces an older one waiting to enter a stage (marked Superseded)	Only the latest change matters
QUEUED (V2)	Executions wait and run one after another, in order; none skipped	Every commit must be deployed, in order
PARALLEL (V2)	Executions run independently at the same time	Feature-branch or PR validation pipelines

Stage conditions, rollback and retry (V2):

- **Automatic rollback:** a stage's on-failure result can be **ROLLBACK** - the stage is re-deployed with the **source revisions of its last successful execution**. Manual stage rollback is also possible.
- **Automatic retry:** on-failure result **RETRY** re-runs the failed actions or the stage.

Condition	When checked	Results
Entry	Before the stage starts	FAIL or SKIP the stage
On success	After the stage succeeds	e.g. ROLLBACK if a rule fails
On failure	After the stage fails	ROLLBACK or RETRY

- **Rule types: CloudWatchAlarm** (roll back if an error alarm fires after deployment), **DeploymentWindow** (deploy only in allowed hours), **VariableCheck**, **LambdaInvoke** (custom check), **Commands**.

5.10 Deploy actions in detail

CloudFormation deploy action:

- `ActionMode`: `CREATE_UPDATE`, `DELETE_ONLY`, `REPLACE_ON_FAILURE` (delete and recreate a failed stack), `CHANGE_SET_REPLACE` (create change set), `CHANGE_SET_EXECUTE` (run it).
- `TemplatePath` = `BuildOutput::template.yml`. `TemplateConfiguration` = file with parameters, tags and **stack policy**. `ParameterOverrides` can use `Fn::GetArtifactAtt` / `Fn::GetParam`.
- `Capabilities`: `CAPABILITY_IAM`, `CAPABILITY_NAMED_IAM`, `CAPABILITY_AUTO_EXPAND` (needed for **SAM templates and macros**).
- `RoleArn` = CloudFormation deployment role. `OutputFileName` writes stack outputs to an artifact.
- **StackSets action:** one template to **many accounts and Regions**.

ECS deploy actions:

- **ECS (rolling update)** needs `imagedefinitions.json`: `[{"name":"container","imageUri":"..."}]`.
- **CodeDeployToECS (blue/green)** needs `imageDetail.json`, `appspec.yaml` and `taskdef.json` (with the `<IMAGE1_NAME>` placeholder).

Other deploy actions:

- **CodeDeploy (EC2 / on-premises):** input artifact contains `appspec.yml` and scripts.
- **Elastic Beanstalk:** deploys the source bundle to an environment.
- **S3 deploy:** `Extract: true` unzips into the bucket (static websites); KMS key, canned ACL, cache-control options.

Lambda Invoke action:

- The function **must call** `PutJobSuccessResult` **or** `PutJobFailureResult`, otherwise the action waits until it times out. The function's role needs those permissions.
- Long jobs: return a **continuation token** so CodePipeline calls the function again.

5.11 Monitoring, notifications and control

- **EventBridge events:** "CodePipeline Pipeline Execution State Change", "Stage Execution State Change", "Action Execution State Change" (STARTED, SUCCEEDED, FAILED, CANCELED, STOPPED, SUPERSEDED, RESUMED). Pipeline FAILED > SNS / Lambda.

- **Notification rules:** pipeline, stage, action and **manual approval needed** events to SNS or Slack / Teams (Chatbot), **no code**.
- **CloudTrail:** who started, stopped, approved, or changed the pipeline.
- **Controls:** start manually (V2: choose source revision and override variables); **stop** ("stop and wait" or "stop and abandon"); **retry** failed stage / action; **disable / enable transitions**; execution history.
- **Pipeline as code:** CloudFormation (`AWS::CodePipeline::Pipeline`) or **CDK Pipelines** (updates itself with a **self-mutation** stage).

5.12 Exam traps: quick table

Scenario	Answer
Start within seconds of a CodeCommit push	EventBridge rule (not polling)
GitHub source without the old OAuth integration	CodeConnections source action
Build needs full Git history (git describe, tags)	CODEBUILD_CLONE_REF + GitPull / UseConnection on the CodeBuild role
S3 source doesn't start the pipeline	Bucket versioning + EventBridge (or CloudTrail) detection
Artifact bucket keeps growing	S3 lifecycle rule on the artifact bucket
Human sign-off before production	Manual approval action + SNS
Only release managers may approve prod	IAM PutApprovalResult on that action's ARN only
Approval left too long	Fails after 7 days
Review infra changes before applying	CHANGE_SET_REPLACE > manual approval > CHANGE_SET_EXECUTE
SAM template fails in CloudFormation action	Add CAPABILITY_AUTO_EXPAND (and CAPABILITY_IAM)
Deploy to several Regions from one pipeline	Cross-Region actions + one artifact bucket and key per Region
Cross-account deploy fails on KMS / decrypt	Customer-managed KMS key; allow target account in key and bucket policies
Tools pipeline deploys CloudFormation into Prod	Cross-account role (action roleArn) + deployment role (RoleArn) + AssumeRole + PassRole
Pass commit ID / image tag to a later action	Variables: namespace + #{Namespace.Var}
Choose environment when starting manually	V2 pipeline-level variables
Monorepo: run only when one folder changes	V2 trigger with file path filter
Release only when a v* tag is pushed	V2 Git tag trigger
Every commit must deploy, in order	V2 QUEUED execution mode
Feature-branch builds must run at the same time	V2 PARALLEL execution mode
Roll back prod when an alarm fires	V2 condition with CloudWatchAlarm rule > ROLLBACK (or CodeDeploy alarm rollback)
Deploy only during business hours	V2 DeploymentWindow rule, or disable the transition
Lambda invoke action hangs until timeout	Function doesn't call PutJobSuccessResult / PutJobFailureResult
ECS rolling deploy fails: missing file	imagedefinitions.json (blue/green: imageDetail.json, appspec.yaml, taskdef.json)
"Approval needed" to Slack without code	Notification rule > Chatbot
Run a fix automatically when the pipeline fails	EventBridge Pipeline Execution State Change FAILED > Lambda / SNS
Same template to many accounts and Regions	CloudFormation StackSets deploy action

5.13 Hands-on lab (about 2.5 hours)

1. Source: your CodeCommit repository, or GitHub through a CodeConnections connection.

2. Create a **V2 pipeline**: Source > Build (CodeBuild) > Approval > Deploy (S3 deploy with Extract, or CloudFormation).
3. Give the source and build actions namespaces; put `#{SourceVariables.CommitId}` and `#{BuildVariables.IMAGE_TAG}` in the approval comment.
4. Manual approval with an SNS email - approve once, reject once.
5. Pipeline variable `TargetEnv` (default `dev`); start manually and override to `test`.
6. GitHub: push trigger for `main` with file path filter `app/**`; change only `README.md` and confirm the pipeline does **not** start.
7. Switch to **QUEUED**, push 3 commits quickly; switch to SUPERSEDED and compare.
8. CloudFormation: CHANGE_SET_REPLACE > approval > CHANGE_SET_EXECUTE.
9. Optional: a deploy action in a second Region; look at the second artifact bucket.
10. EventBridge rule for pipeline FAILED > SNS; break the build on purpose.
11. Clean up: pipeline, artifact buckets, stacks, SNS topic, connection.

5.14 Check yourself

1. A cross-account CloudFormation deploy fails with KMS AccessDenied on the artifact. What do you change?
2. How do you make sure only the release-manager group can approve the production approval action?
3. Monorepo on GitHub: the payments pipeline must run only when `services/payments/` changes. What do you use?
4. Ten commits arrive in 2 minutes and every one must be deployed in order. Which execution mode?
5. Which files does the ECS rolling deploy action need, and which does CodeDeploy blue/green for ECS need?
6. A Lambda invoke action always times out, though the function finishes in 3 seconds. Why?

Answers:

1. Encrypt the artifact bucket with a **customer-managed KMS key**; allow the target account's role in the **key policy** and **bucket policy**.
2. Grant `codepipeline:PutApprovalResult` **only on that approval action's ARN** to the release-manager group, and to nobody else.
3. A **V2 push trigger** with a **file path include filter** `services/payments/**`.
4. **QUEUED** (V2).
5. ECS rolling: **imagedefinitions.json**. Blue/green: **imageDetail.json + appspec.yaml + taskdef.json**.
6. The function never calls **PutJobSuccessResult** (or PutJobFailureResult), so CodePipeline keeps waiting.